

모두를 위한 읽기쉬운 판단



사진 한 장이면 충분합니다.
읽고가 뜻·위험·기한·할 일까지
읽고 정리해드려요.



목차

1	→	팀 소개	6	→	지속 가능성
2	→	문제 정의	7	→	AI 활용 흐름도
3	→	실제 피해 사례	8	→	수익화 구조
4	→	서비스 소개	9	→	실제 시연
5	→	다른 앱과의 차별점			



The Team.

팀 소개

읽고는 같은 문서를 받아도 누군가는 쉽게 이해하고, 누군가는 어려움을 겪습니다. 저희는 그 차이를 기술로 줄이고 싶었습니다.
정보 취약계층도 필요한 정보를 쉽게 이해하고, 중요한 일을 놓치지 않도록 돕는 AI 문서 도우미입니다.

Team.Leader

박소은



- 팀장
- UI / UX 총괄(Figma)

Team.Member

이재희



- 자료 조사
- PPT 제작

Team.Member

노정운



- 앱 개발(Flutter) & Cloud Eng
- 프론트엔드/백엔드 총괄

Team.Member

조인기



- 자료 조사
- PPT 제작

Team.Member

유지환



- 백엔드 개발(Spring boot)
- PPT 제작

Team.Member

황지원



- 자료 조사
- 이미지 UI/UX 제작

Problem

전혀 다른 두 사람, 같은 벽 앞에 서 있다.

베리어트랙

김민수(28세, 경증 발달장애)

원룸 자립 청년. 어려운 고지서들이 오면 서랍에 미룸

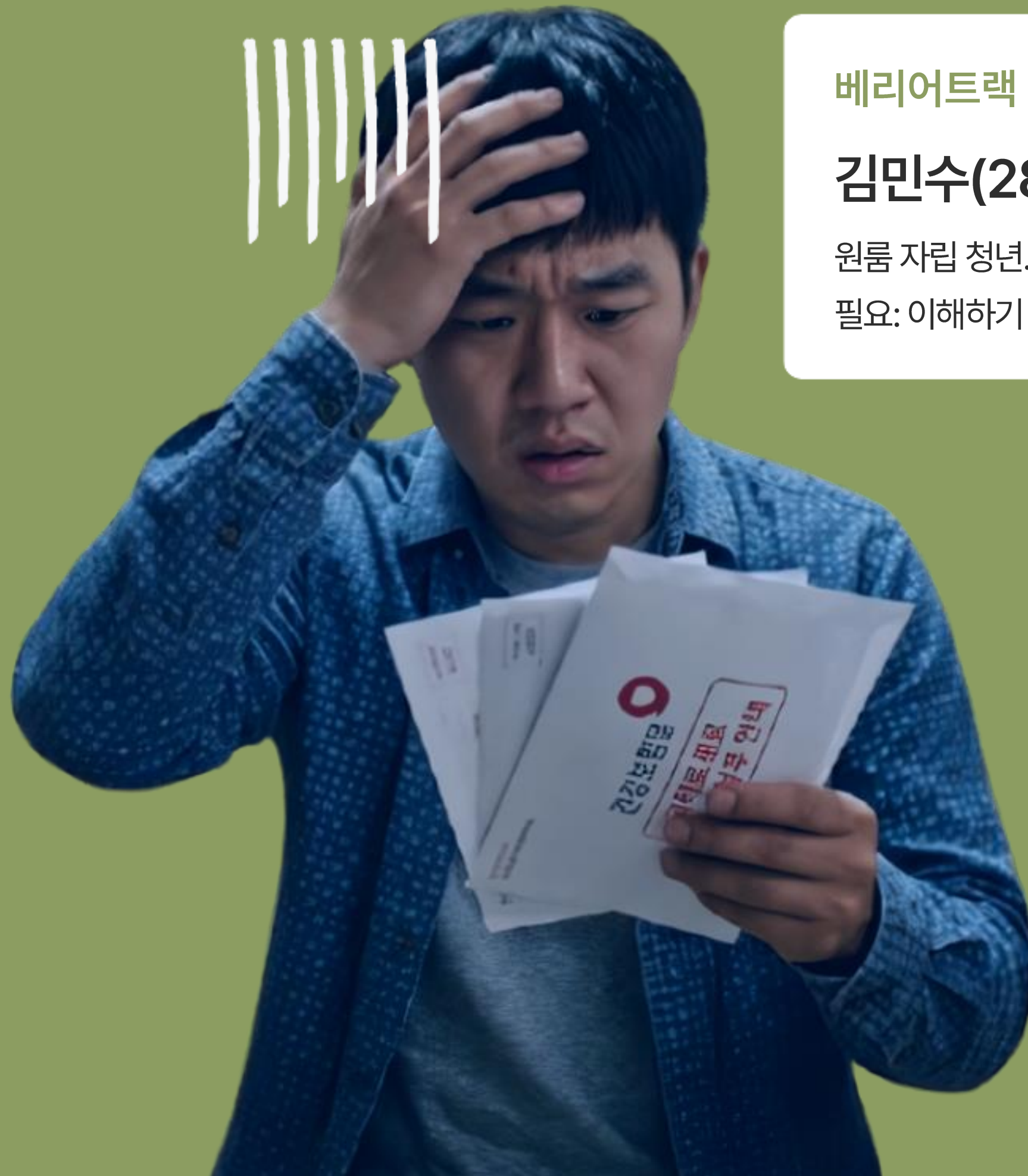
필요: 이해하기 쉬운 한국어 + 할 일 + 캘린더 등록

다문화트랙

스레이몸(35세, 캄보디아 이주노동자)

비닐하우스 노동자, 사장님 카톡에서 어려운 말이 나오면 이해가 잘 되지 않음

필요: 모국어 번역 + 위험감지 + 정중한 예시 답장



Problem.

이건 소수의 문제가 아닙니다.

보건복지부·OECD PIAAC·법무부 공식 데이터(2024-2025)가 보여주는 수치는 충격적입니다.
정보 격차는 개인의 불편함이 아니라, 사기·임금 피해·연체로 이어지는 구조적 취약점 입니다.

발달장애 +
저문해 성인

263만

국내 체류 외국인

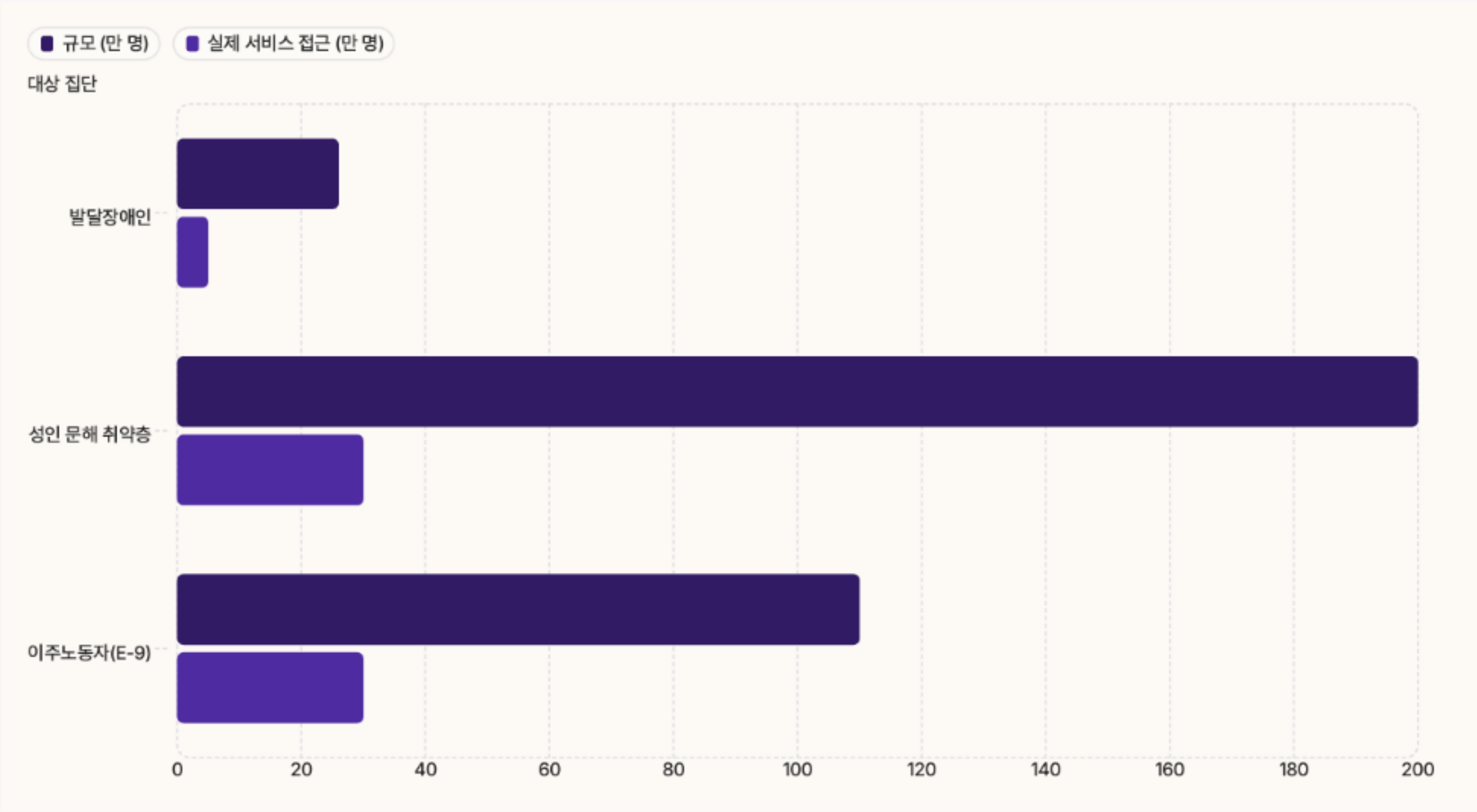
278만

발달장애
서비스 이용률

18%

OECD
문해력 순위

249점 평균 260점



Solution.

문서를 읽지 못하면 피해가 발생한다

발달장애인, 저문해 성인, 이주노동자, 고령층이 공통으로 직면하는 문서 장벽은 단순한 불편함이 아닙니다.
정보 격차는 직접적인 경제적, 안전 피해로 이어집니다

이주노동자
임금체불

연간 약

3,000억원

E-9 비자 이주노동자
(고용노동부, 2024)

이주노동자
산재 발생률

1.7배

제조 농업 이주노동자
(고용노동부, 2024)

권리구제 신청율

12%

이주노동자 전체
(고용노동부, 2024)

보이스피싱 피해액

연간 약

7,700억원

60대 이상 38%
(금융감독원, 2024)

Introduce Service

읽고는 단순한 AI 서비스가 아닙니다

저희 읽고는 단순한 앱이 아니라 정보 접근의 새로운 서비스 입니다.



전체 시장 : 대한민국 (Republic of Korea)

국내 체류 외국인을 포함한 전체 대한민국 인구



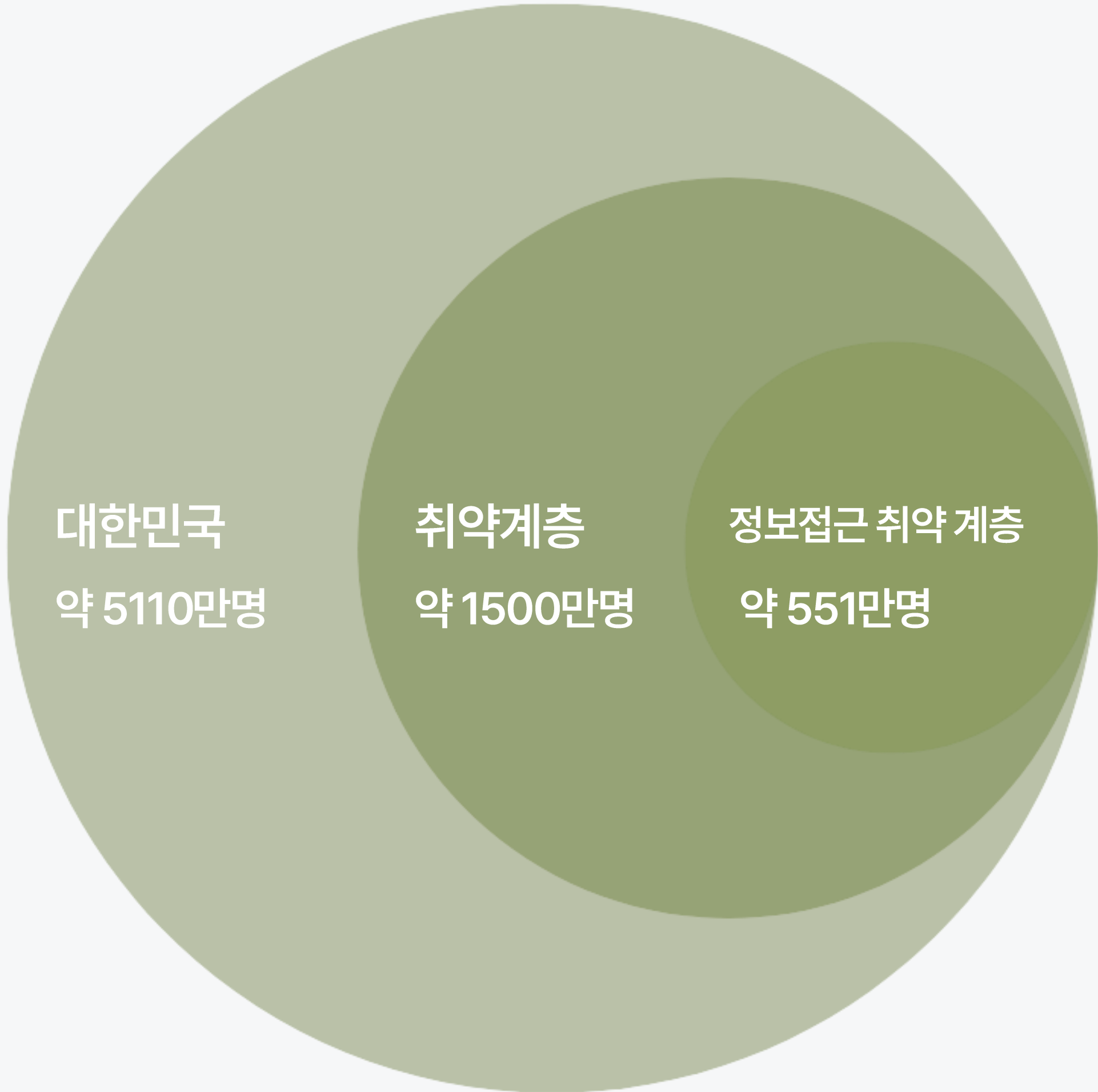
유효 시장 : 취약계층

등록장애인 , 65세 이상 노인 , 국내 체류 외국인을 포함한 취약계층 인구



목표 시장 : 정보접근 취약 계층

발달장애인, 저문해 성인, 국내 체류 외국인(E-9, E-7 등)을 포함한 정보접근 취약계층 인구





이미지 분석

어려운 고지서나 문자를 사진 한 장으로 찍으면, 핵심을 한 문장으로 요약하고 쉬운 말로 풀어 줍니다.

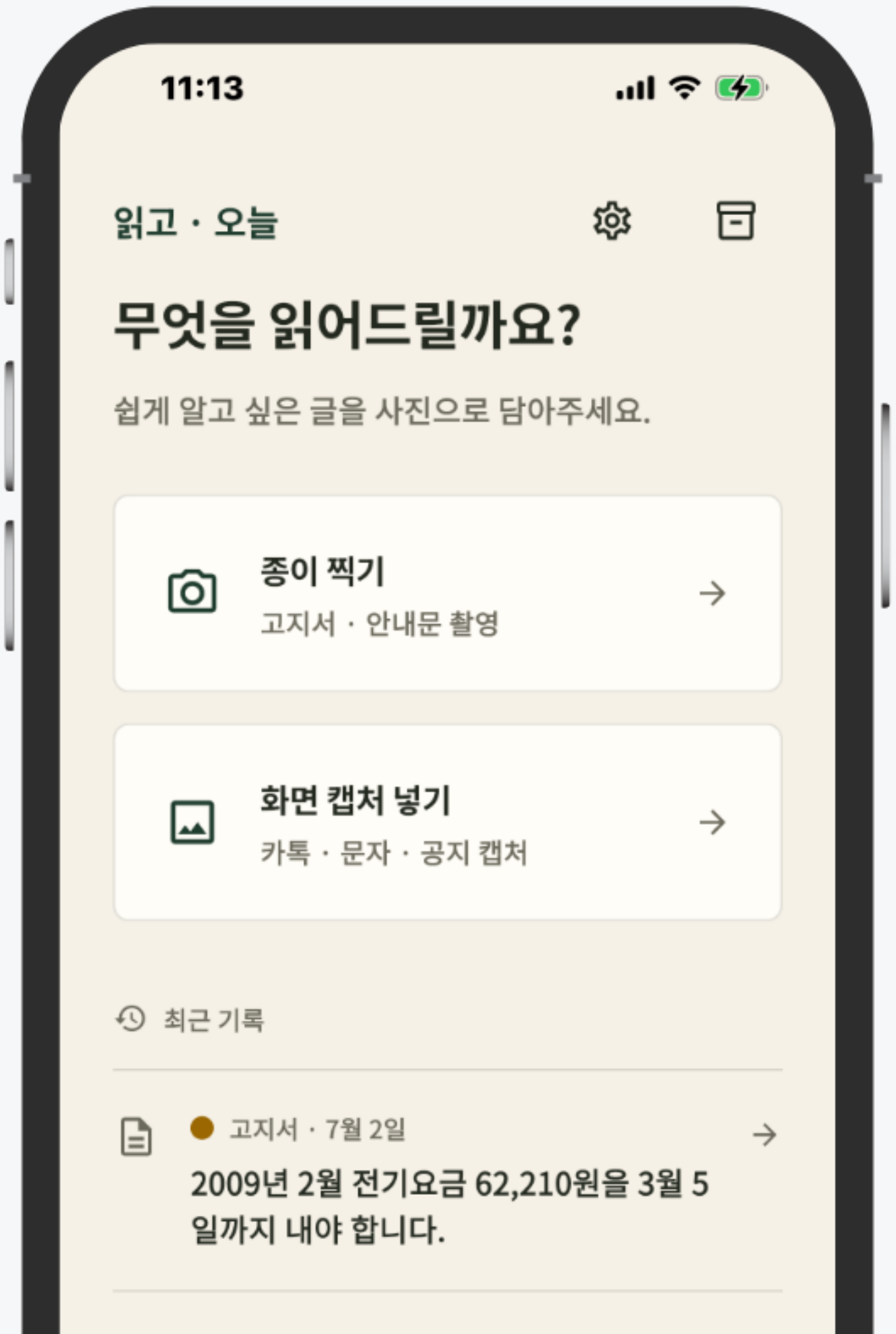


신호등 단계

이 문서가 나에게 위험한지 초록·노랑·빨강으로 알려 줍니다. 사기·임금·기한 같은 손해 신호를 짚어 줍니다.

what

읽고 서비스는 무엇인가



일정 등록(캘린더)

납부 기한이나 마감일을 캘린더에 자동으로 등록해, 깜빡하고 놓치지 않도록 미리 알려 줍니다.



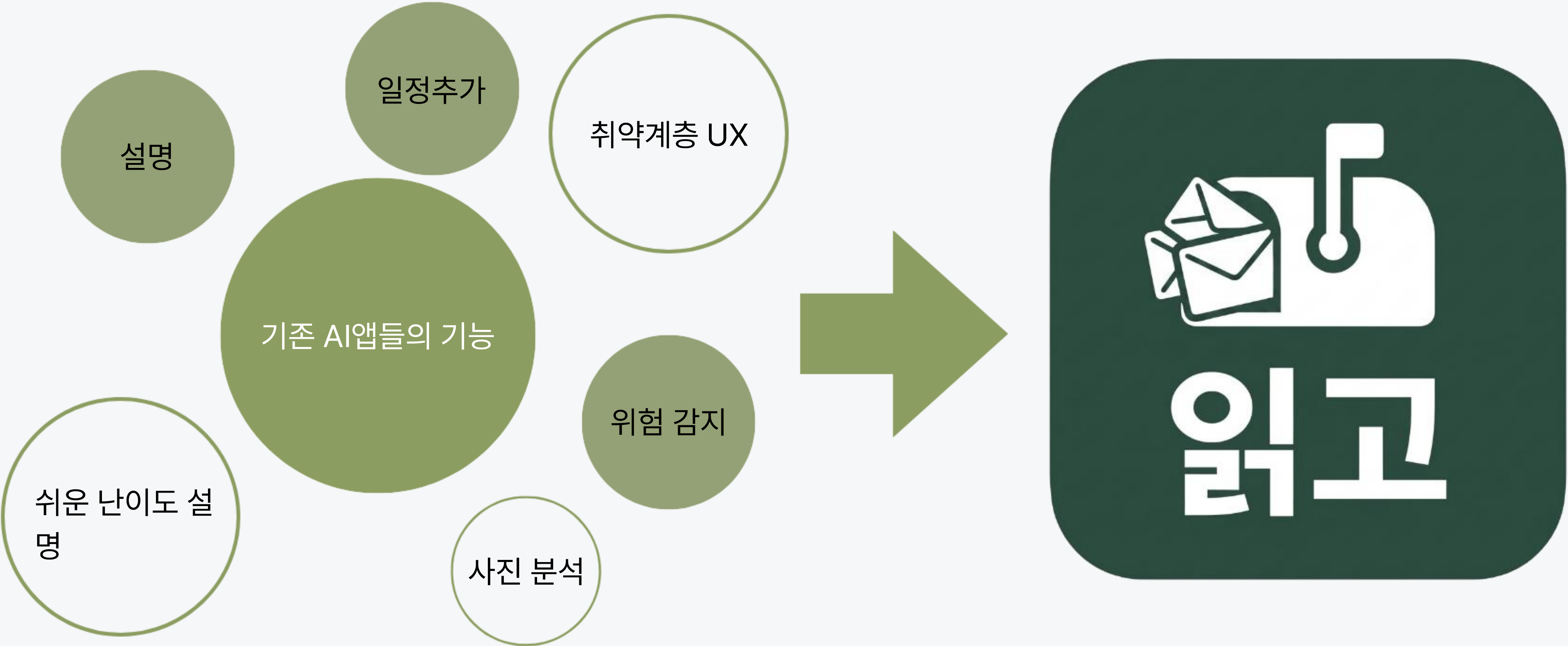
쉬운 난이도의 UI/UX

버튼 하나만 누르면 끝. 글이 어려운 분도 큰 글씨와 쉬운 말로 누구나 혼자 쓸 수 있습니다.

Our Differentiation

다른 앱과의 차별점

저희 읽고는 기존 있던 기능들을 종합적으로 모아서 특정인을 대상으로 편리하게 만든 앱이기 때문에
다른 앱들과 차별화를 두고 있습니다.



sustainability

지속 가능성



POINT 01

매번 달라지는 문서의 뜻과 금액

문서는 익혀도 내용은 매번 새로 옵니다. 읽고는 사진 한 장으로 그 때그때 뜻·금액·기한을 쉬운 말로 정리해, 새 문서가 와도 바로 이해 하게 돕습니다.



POINT 02

한국어를 배워도 생활문서는 여전히 벽

회화가 늘어도 행정·의료·금융 문서는 다른 문제입니다. 읽고는 어 려운 문서를 사용자의 언어와 눈높이에 맞춰 풀어, 배움과 상관없이 매 순간 이해를 돕습니다.



POINT 03

스미싱은 수법이 계속 변화

본문: 사칭·링크·압박 문구는 매번 새 형태로 진화합니다. 읽고는 스 미싱이라고 판단되면 빨간색 경고를 띄우며, 링크를 누르기 전에 멈춰 118 같은 공식 도움처로 확인하도록 안내합니다.



POINT 04

디지털 판단의 어려움

복잡한 문서 앞에서 무엇을 할지 판단하기 어렵습니다. 읽고는 문서 를 버튼 하나로 정리하고, 해야 할 일과 도움받을 곳까지 알려줘 판 단 부담을 덜어 줍니다.

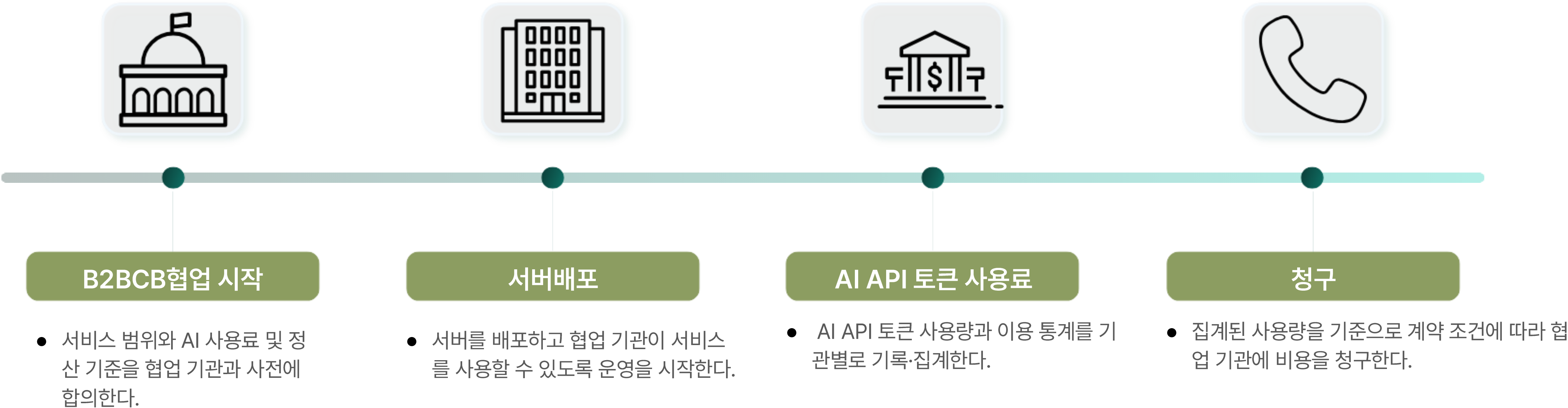
KISA Insight 2024 Vol.07, 국내·외 피싱 대응 현황 및 시사점

교육부, 제1차 성인디지털문해능력조사 결과 발표

Business Model

수익화 구조

B2BCB 협업 운영 프로세스



Business Model

B2BCB 협업 대상

저희 읽고는사용자는 무료, 국가.기관이 비용을 부담합니다



공공기관 구독

- 지자체, 복지관, 장애인복지센터, 다문화가족지원센터 등이 읽고 서비스를 도입해 이용료를 지불합니다.



금융기관

- 은행이나 보험사가 고지서, 약관, 금융 안내문을 쉽게 설명하는 서비스로 읽고를 도입합니다.
- 금융사는 고객 서비스 향상을 위해 사용료를 지불합니다.



통신사·플랫폼 제휴

- 통신사나 복지 플랫폼이 읽고 기능을 자사 앱에 탑재하고 라이선스 비용을 지급하는 방식입니다.

AI Workflow

AI 활용 흐름도

읽고는 AI로 만들어진 앱이자, AI로 작동하는 앱입니다.
제품의 심장과 개발 과정 전체, 두 곳에 AI를 썼습니다.

① 제품 속 AI — 멀티모달 LLM 분석 엔진

② 개발 속 AI — Claude Code 에이전트 하네스

AI Workflow

AI를 두 번 썼습니다

다른 팀이 목업으로 보여줄 때, 저희는 실제 구동 앱과 라이브 AI로 보여드립니다.

PRODUCT · 제품 엔진

멀티모달 AI가 곧 서비스

사진 한 장을 AI가 직접 읽고, 뜻·위험·기한·할 일을 구조화된 JSON 한 번에 돌려줍니다.
OCR 없이 이미지에서 바로 이해합니다.

- 이미지 → LLM 1회 호출 → 결과 카드 (라이브 시연)
- 쉬운 한국어 / 영어 모드 — 파라미터 분기 하나로 두 사용자 지원
- 신호등 위험 판정 · 할 일 · 답장 초안 · 캐릭터 표정까지 AI가 결정

PROCESS · 개발 하네스

Claude Code 에이전트가 팀원

기획서 작성부터 Flutter 앱·Spring 백엔드 구현, 코드 리뷰까지 AI 코딩 에이전트 하네스를
구축해 5명 팀이 3일 만에 실서비스를 완성했습니다.

- 문서(PRD·API명세) → 작업지시서 → 서브에이전트 병렬 개발
- Figma MCP로 디자인 시안을 코드로 핸드오프
- 목데이터 우선 개발 → 한 줄 스왑으로 실서버 전환

심사 기준 “AI 툴 활용도 20점” — 도구 사용을 넘어, 제품 코어와 개발 공정 전체가 AI 위에서 돌아갑니다

Product Pipeline

사진 한 장이 카드가 되기까지

앱은 AI를 모릅니다. 서버가 모델을 대신 부르고, 앱은 완성된 답만 받습니다.



모드는 파라미터 하나로 분기

output_mode=easy_korean → 김민수에게 쉬운 한국어,
output_mode=native · lang=en → 마리아에게 영어. 같은 엔진, 같은 호출로 배리어프리·다문화 두 트랙을 동시에 해결합니다.

왜 “1회 호출”인가

OCR → 번역 → 요약을 따로 부르면 느리고 비싸고 실패 지점이 늘어납니다. 멀티모달 호출 한 번에 읽기·풀이·위험판정·할 일 추출을 끝내 **속도·비용·안정성**을 모두 잡았습니다.

Preview

실제 시연



<https://ilgo-app.netlify.app/>

Prompt Harness

AI가 마음대로 말하지 못하게 — 답을 강제하는 설계

취약계층에게 틀린 답은 불편이 아니라 피해입니다. 프롬프트를 '계약'으로 만들었습니다.

① JSON 스키마 강제

응답이 `"{"` 로 시작하도록 강제(prefill·구조화 출력 모드)해 JSON 외의 답을 원천 차단.
temperature 0.2로 형식 이탈 최소화. 파싱 실패 시 자동 재시도 → 그래도 실패하면 `422` 로
명확히 격리.

② 사실 통제 규칙 (시스템 프롬프트)

“금액·날짜는 원문 그대로 인용 / 불확실하면 null / 없는 사실 생성 금지.” 환각을 스키마 수준에
서 봉쇄합니다.

③ 안전 편향 — 확신 없으면 노랑

●●● 위험 판정이 애매하면 무조건 yellow로 상향 — 사기를 '안전'으로 놓치는 false
negative를 구조적으로 방지. 스미싱·임금차감·부당공제·과태료·신고기한을 위험 유형으로 명시.

④ 실패까지 설계 — 모델 이중화

메인 `gemini-3-flash-preview` 장애 시 gpt-5.4로 자동 전환, 그마저 실패(`502`)하면
폴백 캐시 + '예시' 배지로 응답 — 발표 데모가 절대 멈추지 않습니다.

```
// 프론트·백엔드가 공유하는 데이터 계약 — ReadResult
{
  "doc_type": "notice",
  "summary_one_line": "2월 전기요금 62,210원을  
3월 5일까지 내야 합니다.",
  "explained_text": "...쉬운 말 풀이...",
  "cards": { "what", "when", "where",  
            "amount", "deadline" },
  "risks": [{ "level": "green|yellow|red",  
              "type", "message" }],
  "actions": [{ "text", "priority" }],
  "reply_drafts": [{ "korean", "note_in_lang" }],
  "consequence": "안 내면 연체료가 붙어요."
}
```

이 스키마 하나가 프롬프트 출력 = API 응답 = Flutter freezed 모델로 그대로 이어집니다. AI
출력이 곧 타입 안전한 코드가 되는 구조.

Character AI

또바기 — 표정과 위로까지 AI가 고릅니다

숫자와 경고만으로는 부족합니다. 같은 분석 호출 안에서 AI가 감정 신호를 함께 결정합니다.

**warn**

스미싱·사기 감지
“이 링크 누르지 마세요!”

**caution**

기한·금액 주의
“3월 5일까지 잊지 마요.”

**safe**

안전한 문서
“이건 안전해 보여. 걱정 마.”

**victim**

이미 피해 정황
“늦지 않았어요. 지금 바로
112에 신고해요.”

추가 호출 없이, 같은 응답 안에서

분석 시스템 프롬프트에 **character_state** 선택 기준을 넣어, 문서 이해와 감정 신호 결정이 **단일 호출**로 끝납니다. 답장 초안의 톤 바꾸기(정중하게/짧게)만 별도 재호출.

AI 출력을 절대 그대로 믿지 않기

서버가 이상한 값을 보내면 프론트가 위험도 기반 **표정으로 자동 폴백**. enum 4개 외 값은 무시. AI가 실수해도 사용자 화면은 항상 올바릅니다.

Ideation

아이디어 회의 — 30개를 1개로 좁힌 적대적 채점

제주 현장 관찰과 팀 회의에서 나온 아이디어를 AI가 발산시키고, 다시 AI가 공격했습니다.



후보 발산

팀 회의 × Claude 브레인스토밍
트랙별 6~9개, 총 ~30개



적대적 채점

자체 110점 루브릭으로
AI가 모든 후보를 공격·채점



트랙 우승작

쉬운고지서 106점
오뮴(오늘 뭐래?) 103점



통합 통찰

“걸은 다른 앱,
속은 같은 엔진”



읽고 탄생

배리어프리 + 다문화
한 엔진으로 동시 출전

채점표부터 심사기준으로 설계

사회가치(30)·창의(25)·AI활용(20)에 2일 실현성(20)·데모 임팩트(15)를 더한 110점 루브릭을 만들어, AI가 심사위원 입장에서 후보를 떨어뜨리게 했습니다. 감이 아니라 점수로 결정.

회의록이 곧 프롬프트

관찰 메모와 회의 내용을 전부 문서로 만들어 AI에 투입했고, 탈락 이유와 트랙 전환 시나리오(flip note)까지 문서로 남겨 언제든지 결정을 재검토할 수 있게 했습니다.

Plan Review

코드보다 먼저, 에이전트 126개가 기획을 심사했습니다

gstack의 **plan-ceo-review** 스킴을 Opus 4.8 + ultracode 멀티에이전트 오케스트레이션으로 실행.

126개

투입된 서브에이전트
총합

82개

동시 의견 제시
(다관점 리뷰 패널)

40개

교차 검증
(적대적 반박)

4개

정밀 채점
(심사기준 루브릭)

사람 한 명이 못 보는 각도를 병렬로

82개 에이전트가 창업자·심사위원·김민수·마리아·엔지니어 등 서로 다른 시점에서 **동시에** 기획을 뜯었고, 40개 검증 에이전트가 근거 없는 지적을 걸러낸 뒤, 4개가 심사기준(사회가 치·창의·AI활용·완성도)으로 최종 채점했습니다.

결과 — 구멍은 코드 전에 메운다

이 리뷰에서 나온 지적이 게스트 모드, 분석 실패 폴백, 예시 체험, 위험별 도움 CTA 같은 개선으로 이어졌습니다. 사람 CEO 리뷰와 합쳐 **이중 심사망** — AI가 만들고 AI가 먼저 떨어뜨립니다.

Dev Harness

개발 하네스 — AI 에이전트 팀으로 만든 앱

Claude Code(AI 코딩 에이전트)에게 '기억·설계도·작업지시'를 주는 하네스를 만들어 개발 속도를 끌어올렸습니다.



SKILL 01

Figma MCP 디자인 핸드오프

Figma 시안을 MCP로 에이전트가 직접 읽어 Flutter 위젯으로 변환 — 디자이너와 개발 사이 전달 손실 제거.

SKILL 02

Mock-first 개발

목 JSON으로 전 화면 완성 후 `USE MOCK=false` 한 줄로 실서버 스왑(Repository 인터페이스 + DI). 백엔드를 기다리지 않는 병렬 공정.

SKILL 03

코드 생성 자동화

freezed + json_serializable로 ReadResult 스키마에서 모델·직렬화 코드 자동 생성 — AI가 정의한 계약이 그대로 컴파일 타임 안전성이 됩니다.

Orchestration

세 AI가 한 팀 — 지휘하고, 검증하고, 잇습니다

한 모델에 다 맡기지 않았습니다. 회사가 다른 AI 셋에게 역할을 나누고, 도구 연결은 전부 MCP로 표준화했습니다.

VERIFY

Codex — 코드 검증

Claude가 만든 코드를 다른 회사 AI가 교차 리뷰. 같은 모델은 같은 맥락을 공유하기 때문에, 검증자는 일부러 다른 두뇌를 씁니다.

ORCHESTRATOR

Claude — 메인 오케스트레이터

작업 분배·작업지시서 발행·서브에이전트 지휘·결과 통합. ralph 루프와 126개 리뷰 패널도 Claude가 지휘했습니다.

DESIGN

Gemini(agy) — 디자인 연동

Figma MCP를 통해 디자인 시안을 직접 읽고 Flutter 위젯으로 핸드오프. 디자이너와 개발 사이 전달 손실 제거.

 Figma MCP — 디자인이 곧 입력

스크린샷 복붙이 아니라 프로토콜로 시안 데이터를 읽습니다. 색·간격·컴포넌트 구조가 그대로 코드 근거가 됩니다.

 Supabase MCP — DB도 대화로

스키마 확인·마이그레이션·데이터 점검을 에이전트가 MCP로 직접 수행 — ERD 문서와 실제 DB의 불일치를 즉시 잡아냅니다.

역할 분리(만드는 AI ≠ 검증하는 AI) × MCP 표준 연결 — 오케스트레이션이 곧 품질 장치

Skill Stack

Claude Code 위에 얹은 오픈소스 스킬 스택 4종

맨몸의 AI가 아니라, 검증된 스킬 프레임워크를 조합해 '일하는 방식' 자체를 설계했습니다.

PROCESS

superpowers

브레인스토밍 → TDD → 체계적 디버깅
→ **완료 전 검증**까지, 행동하기 전에 반드시 절차 스킬을 거치게 강제하는 규율 레이어.

github.com/obra/superpowers

REVIEW

gstack

plan-ceo-review 등 리뷰 스킬
모음. 기획·디자인·QA를 창업자 시선으로 심사 — 앞 장의 126개 에이전트 리뷰가 이 스킬입니다.

github.com/garrytan/gstack

EXECUTION

oh-my-claudecode

멀티에이전트 실행 엔진. **ralph** 루프·에이전트 팀·자동 QA 사이클로 “될 때까지”를 시스템으로 만듭니다. (다음 장)

github.com/yeachan-heo/oh-my-claudecode

CONTEXT

caveman

응답·서브에이전트 출력을 압축해 토큰을 ~75% 절감. 긴 개발 세션에서도 컨텍스트가 마르지 않게 하는 절약 레이어.

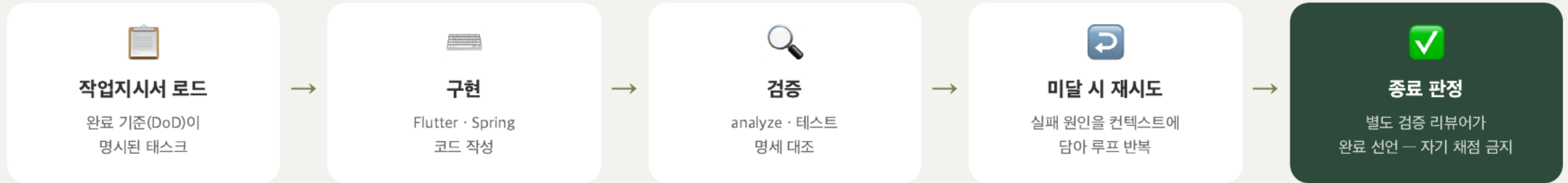
github.com/juliusbrussee/caveman

규율(superpowers) × 심사(gstack) × 실행(omc) × 절약(caveman) — 네 층이 곧 저희의 개발 하네스입니다

Coding Loop

구현은 ralph 루프 — 끝을 알고, 끝날 때까지 돕니다

oh-my-claudecode의 ralph는 완료까지 스스로 도는 실행 루프. 루프의 최대 적인 ‘모호함’을 컨텍스트 엔지니어링으로 제거했습니다.



종료조건을 코드처럼 명시

“flutter analyze 0건 · 테스트 전부 통과 · API 명세 일치”를 작업지시서에 박아, 루프가 무한히 돌거나 어중간하게 멈추는 일을 없었습니다.

컨텍스트 엔지니어링

CLAUDE.md(프로젝트 두뇌) + 문서 인덱스 + caveman 압축으로, 긴 루프에서도 에이전트가 **같은 사실 위에서** 일하게 유지 — 모호한 지시가 곧 버그이기 때문입니다.

구현과 심판의 분리

코드를 짰 에이전트가 아닌 **별도 리뷰어 에이전트**가 종료를 판정합니다. “내가 보기엔 다 됐어요”를 시스템 차원에서 금지.

Quality Loop

AI가 만들고, 사람이 심사하고, AI가 고칩니다

에이전트 산출물을 그대로 믿지 않고, 리뷰 → 문서화 → 재작업의 루프를 돌렸습니다.

1회

문서 분석당
LLM 호출 수

0건

flutter analyze
이슈

9/9

테스트
통과

2트랙

배리어프리 + 다문화
한 엔진으로 동시 대응

사람의 리뷰가 방향을 잡습니다

CEO 기능 리뷰에서 나온 지적을 문서로 만들어 에이전트에 재투입 — 게스트 모드, 분석 실패 폴백, 예시 체험, D-day 배지, 답장 톤 재생성 등 **10개 개선을 하루에 반영**했습니다.

데모 안전망도 AI 산출물로

개발용 목 JSON을 **라이브 실패 시 폴백 캐시**로 재활용. 오프라인 비상 빌드 (**USE MOCK=true**)까지 준비되어 시연은 어떤 상황에도 진행됩니다. 실서버는 Railway 에 배포, JWT 로그인 연동 완료.

Responsible AI

책임 있게 쓰기 위한 네 가지 약속

POINT 01

마지막 판단은 사람이

AI의 답은 참고일 뿐, 중요한 결정은 항상 사람이 확인합니다. 위험한 문서는 112·118 같은 공식 기관으로 안내합니다.

POINT 02

개인정보는 모으지 않기

찍은 사진은 분석이 끝나면 저장하지 않는 것이 기본입니다. 꼭 필요한 정보만, 꼭 필요한 만큼만 다룹니다.

POINT 03

모르면 모른다고 하기

확실하지 않은 위험은 ‘안전’이라 말하지 않고 ‘주의’로 알립니다. 없는 내용은 지어내지 않도록 설계했습니다.

POINT 04

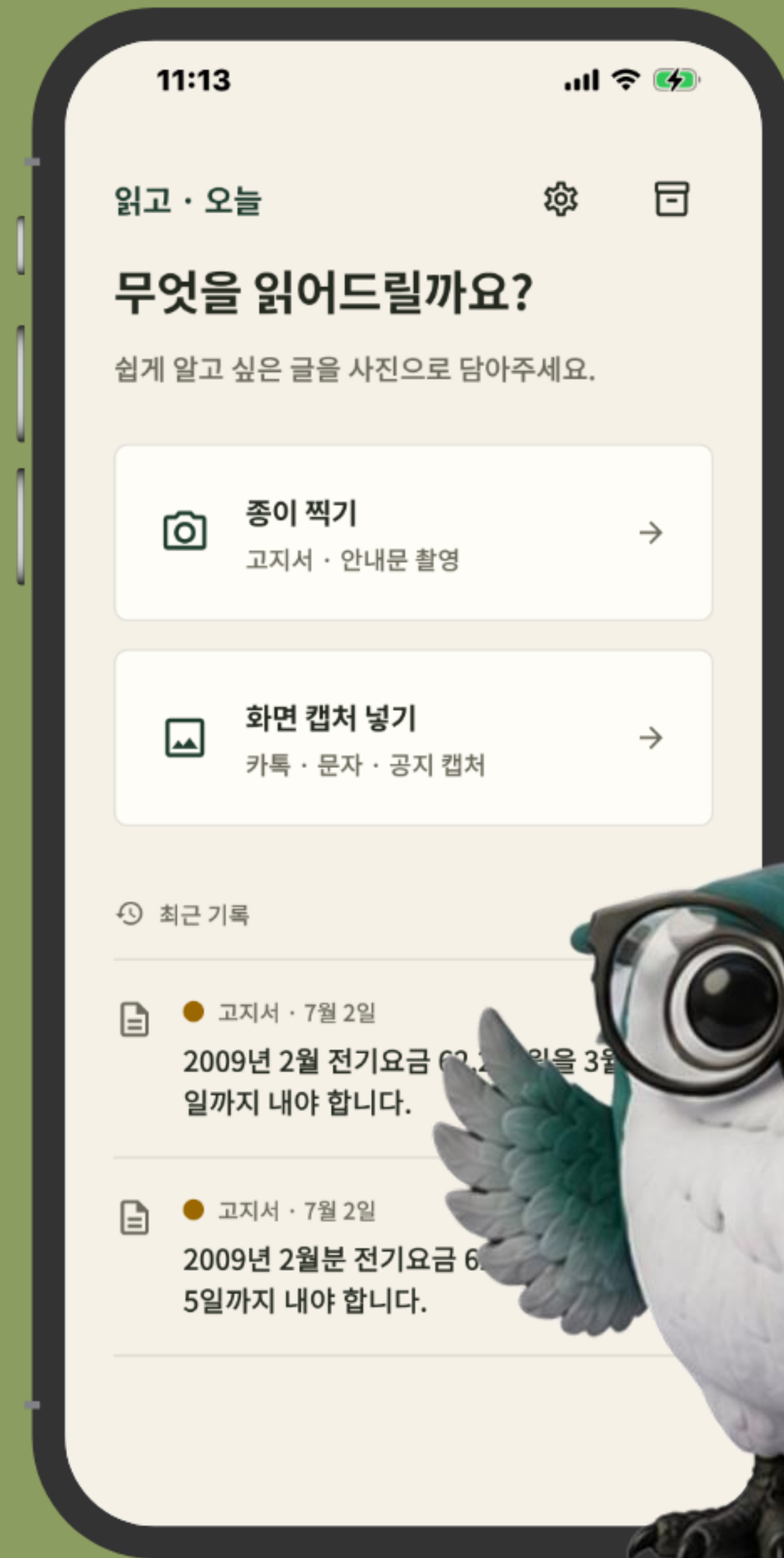
소외된 사람 먼저

큰 글씨, 쉬운 말, 음성 안내 — AI의 혜택에서 밀려나기 쉬운 사람을 첫 번째 사용자로 생각하고 만들었습니다.

“기술은 모두를 위한 것이어야 합니다.”

AI를 가장 필요한 곳에, 가장 신중하게 — 팀 읽고

같은 문서를 받아도
누군가는 쉽게 이해하고,
누군가는 어려움을 겪습니다.
저희는 그 차이를
기술로 줄이고 싶습니다.



Q&A